



Topics in Software Engineering – Software Reengineering

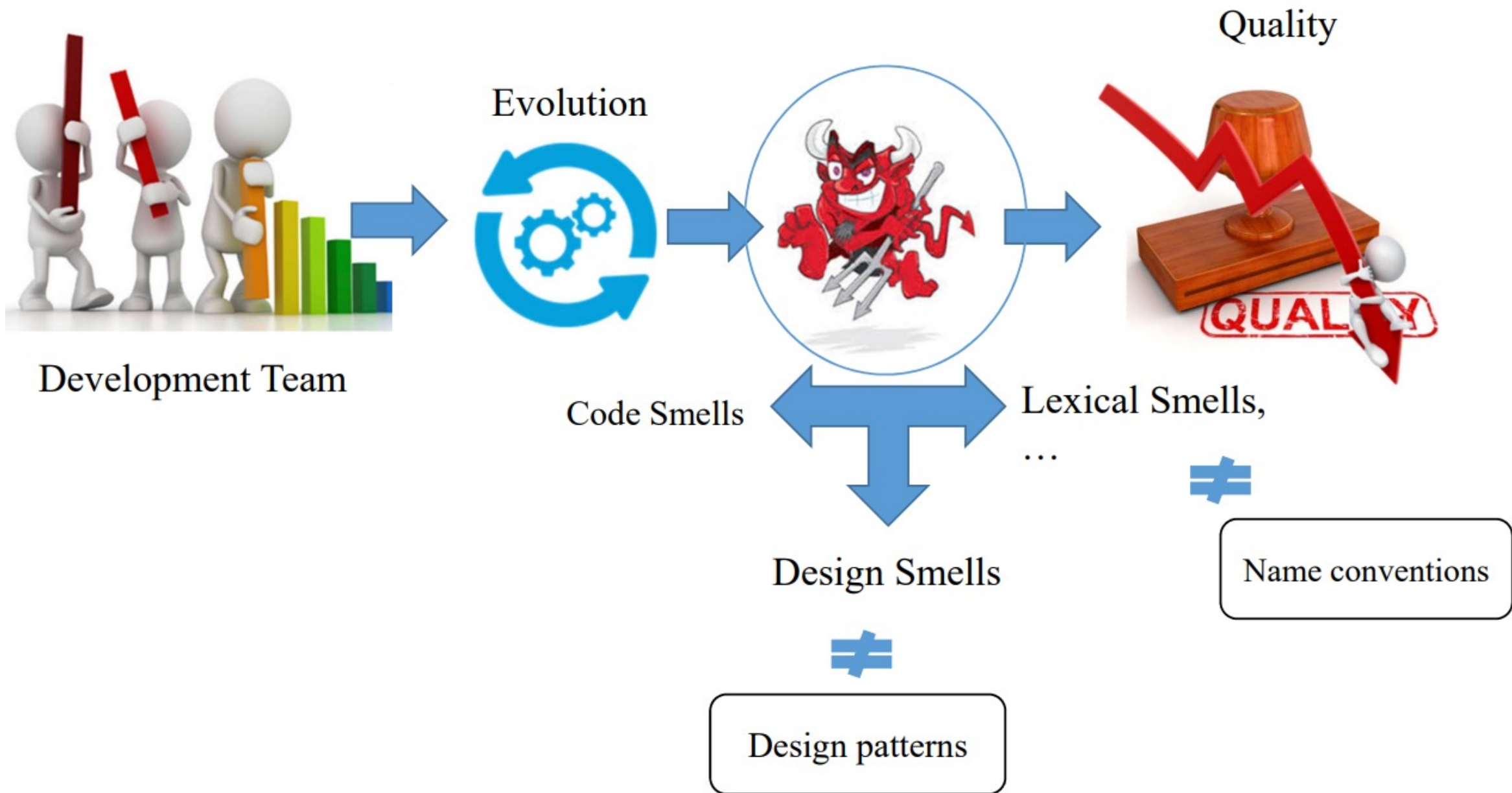
By

Dr. Junaid Akram

Assistant Professor, Department of Computer Science COMSATS (Lahore)

Software Maintenance

(Code Smells)



Smells

Development team may implement software features with poor design, or bad coding...

- Code Smells (Low level (local) problems)
 - ✓ Poor coding decisions
- Lexical smells (Linguistic Anti-patterns)
 - ✓ Poor practice in the naming, documentation, ... in the implementation of an entity.
- Anti-patterns (High Level (global) problems)
 - ✓ Poor design solutions to recurring design problems



Outline

Concepts and Definitions

Examples of Code smells

Research on code smells

State of the Art

Code smells detection techniques

Evaluation of technique

Conclusion

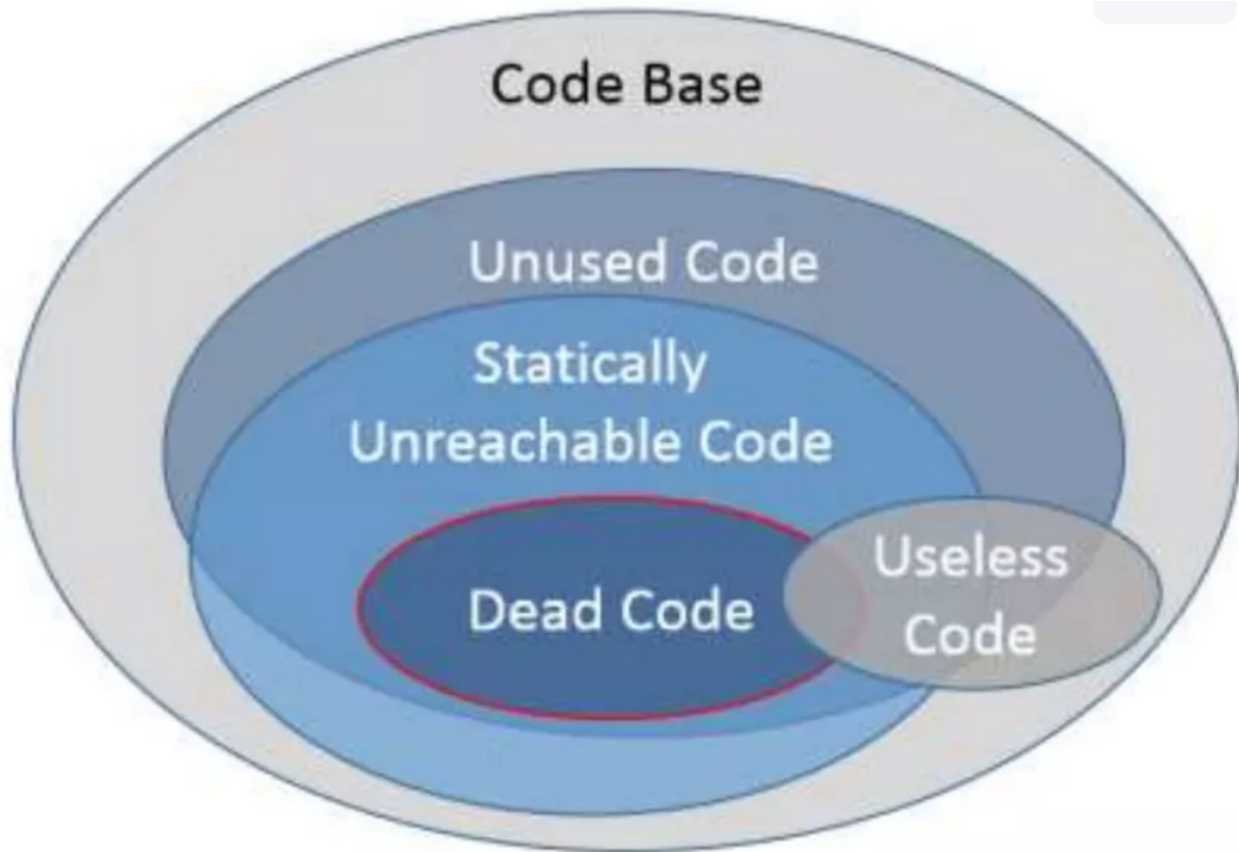
Code Smells

Flower et al[4] identified 22 code smells in 1999.

Code Smells are usually implementation problems.

It is claimed that classes that are involved in certain code smells are liable to be changed more frequently and have more defects than other classes in the code[2]

God Class, Data Class, Long method parameters, Feature envy etc. are code smells



Code Smells Definitions

Code smell is any symptom that indicating something wrong

Code smell is actually a hint that something has gone wrong somewhere in your code which may cause problems

Code Smells are structural characteristics of software that may indicate a code or design problem that makes software hard to evolve and maintain, and may trigger refactoring of code

Why Cod Smells Occur?



Frequent changes and enhancement



Violation of standard design and coding principles



Deadline Pressure



Priority to features over quality



Inexperience of developers



In-appropriate Refactoring operations



Poor human resource planning



Framework constraints etc.

Motivation for Code Smells

- Successful systems are maintained over multiple years
- Exceeding size and complexity are problem for evolution and maintenance
- System's Life-Cycle Properties worsen over time
 - Understandability
 - Testability
 - Extensibility
 - Reusability
- Restructuring/Refactoring helps to improve life-cycle properties
 - Code Smells

Summary of Code Smells

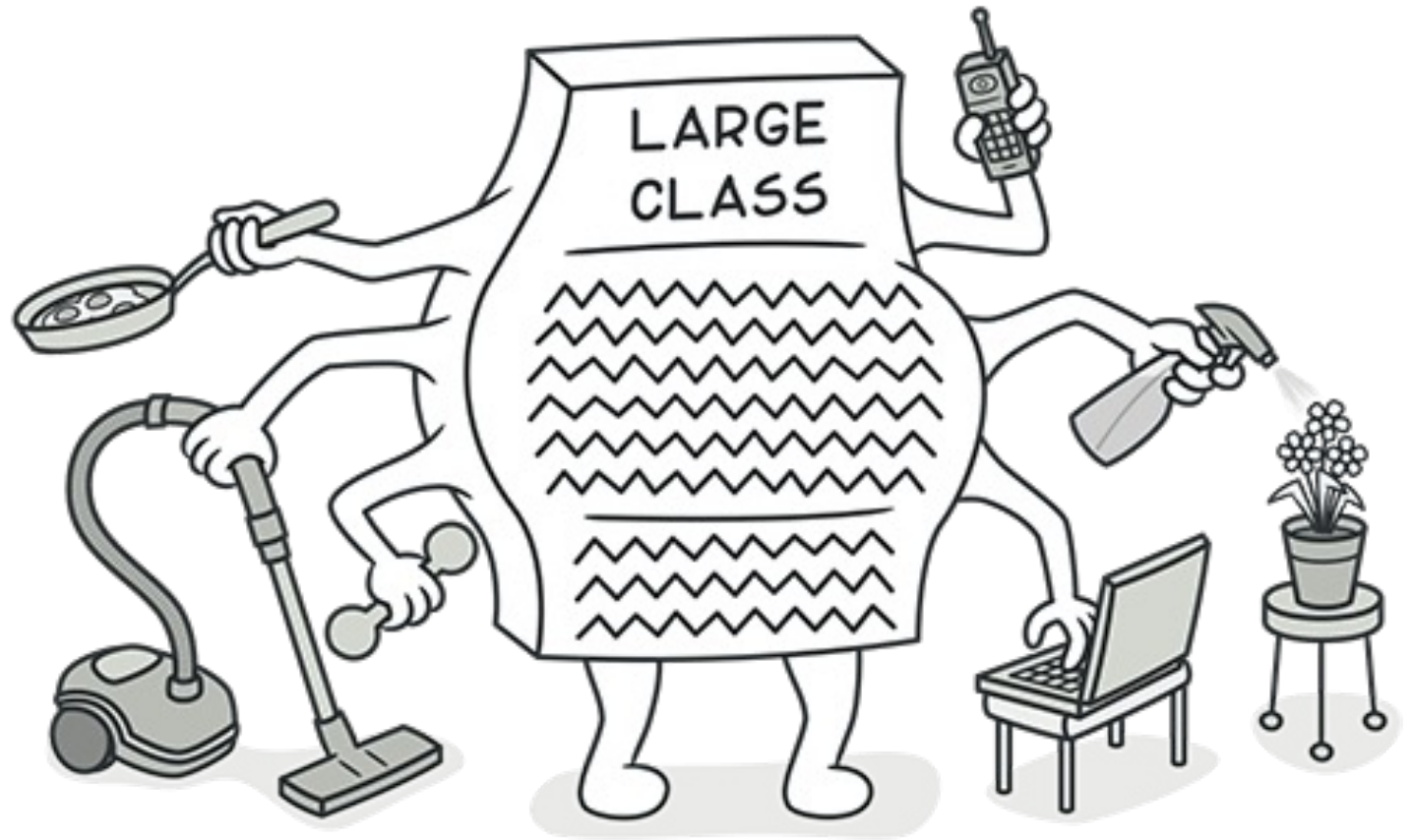
Smell	Description
<i>Duplicated Code</i>	Consists of equal or very similar passages in different fragments of the same code base
<i>Long Method/Long Function</i>	Very large method/function and, therefore, difficult to understand, extend and modify. It is very likely that this method has too many responsibilities, hurting one of the principles of a good OO design (SRP: <i>Single Responsibility Principle</i> (Martin and Martin, 2007))
<i>Large Class</i>	Class that has many responsibilities and therefore contains many variables and methods. The same SRP also applies in this case
<i>Long Parameter List</i>	Extensive parameter list, which makes it difficult to understand and is usually an indication that the method has too many responsibilities. This smell has a strong relationship with <i>Long Method</i>
<i>Divergent Change</i>	A single class needs to be changed for many reasons. This is a clear indication that it is not sufficiently cohesive and must be divided
<i>Shotgun Surgery</i>	Opposite to <i>Divergent Change</i> , because when it happens a modification, several different classes have to be changed
<i>Feature Envy</i>	When a method is more interested in members of other classes than its own, is a clear sign that it is in the wrong class
<i>Data Clumps</i>	Data structures that always appear together, and when one of the items is not present, the whole set loses its meaning
<i>Primitive Obsession</i>	It represents the situation where primitive types are used in place of light classes
<i>Switch Statements/Repeated Switches</i>	It is not necessarily smells by definition, but when they are widely used, they are usually a sign of problems, especially when used to identify the behavior of an object based on its type
<i>Parallel Inheritance Hierarchies</i>	Existence of two hierarchies of classes that are fully connected, that is, when adding a subclass in one of the hierarchies, it is required that a similar subclass be created in the other
<i>Lazy Class</i>	Classes that do not have sufficient responsibilities and therefore should not exist
<i>Speculative Generality</i>	Code snippets are designed to support future software behavior that is not yet required
<i>Temporary Field</i>	Member-only used in specific situations, and that outside of it has no meaning
<i>Message Chains</i>	One object accesses another, to then access another object belonging to this second, and so on, causing a high coupling between classes

<i>Middle Man</i>	Identified how much a class has almost no logic, as it delegates almost everything to another class
<i>Inappropriate Intimacy</i>	A case where two classes are known too, characterizing a high level of coupling
<i>Alternative Classes with Different Interfaces</i>	One class supports different classes, but their interface is different
<i>Incomplete Class Library</i>	The software uses a library that is not complete, and therefore extensions to that library are required
<i>Data Class</i>	The class that serves only as a container of data, without any behavior. Generally, other classes are responsible for manipulating their data, which is a case of <i>Feature Envy</i>
<i>Refused Bequest</i>	It indicates that a subclass does not use inherited data or behaviors
<i>Comments</i>	It cannot be considered a smell by definition but should be used with care as they are generally not required. Whenever it is necessary to insert a comment, it is worth checking if the code cannot be more expressive
<i>Mysterious Name</i>	Non-significant names that do not represent the software elements
<i>Global Data</i>	It can be modified from anywhere in the code base, and there's no mechanism to discover which bit of code touched it
<i>Mutable Data</i>	It changes to data can often lead to unexpected consequences and tricky bugs
<i>Lazy Element</i>	Software elements designed to grow, but do not conform with software evolution
<i>Insider Trading</i>	Coupling problems caused by trade data between modules

List of code smells presented by [Wake \(2003\)](#).

Smell	Description
<i>Type Embedded in Name</i>	Names used, usually defined with duplication, such as <i>schedule.addCourse(course)</i> instead of <i>schedule.add(course)</i> . This category also included the use of Hungarian notation and variables that reflect their type in counterpoint to their purpose or function
<i>Uncommunicative Names</i>	Names used in software elements (usually attributes and local variables) that do not communicate their name/intent enough, such as <i>x</i> or <i>value1</i> . It is even more critical when used in methods and classes
<i>Inconsistent Names</i>	Same name used in different places, for different purposes
<i>Dead Code</i>	Characterized by a variable, attribute, or code fragment that is not used anywhere. It is usually a result of a code change with improper cleaning
<i>Null Check</i>	Occurrences that repeatedly appear, verifying the null values of objects
<i>Complicated Boolean Expression</i>	Code snippets involving boolean operators such as <i>and</i> , <i>or</i> and <i>not</i>
<i>Special Case</i>	Complex conditional statements
<i>Magic Numbers</i>	Numeric values that appear deliberately in the code and that invariably do not change

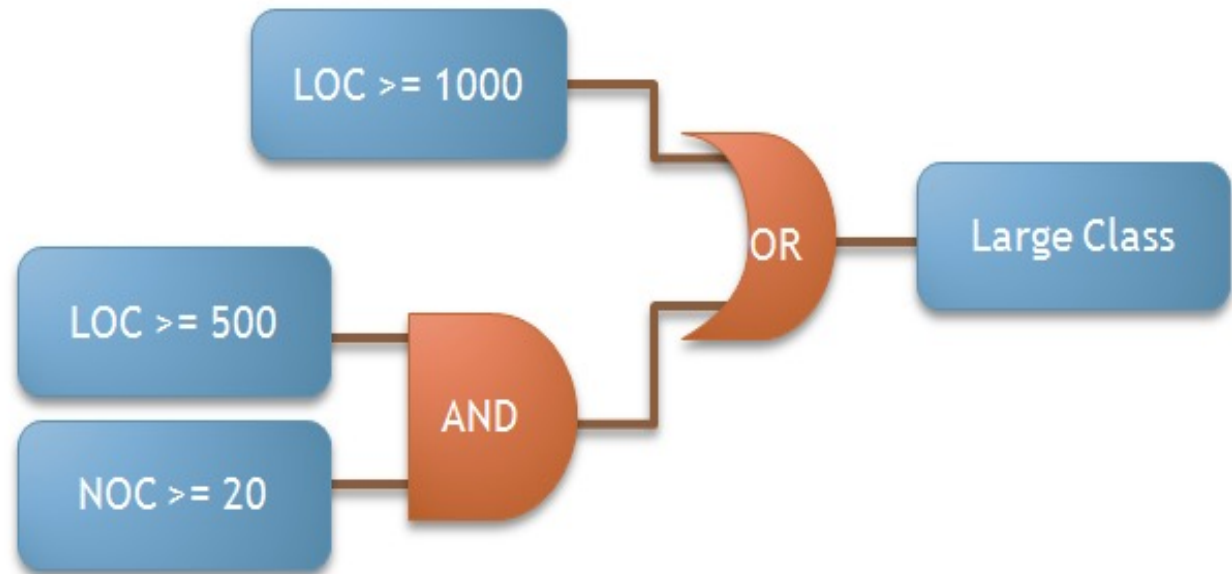
Large Class



A class is trying to do too much. These classes have too many instance variables or methods.

Large Class

- A large class has monopoly on entire application
- Some methods belong to other classes



Large Class

Development AntiPattern:

The Blob

- **Symptoms**
 - Single class with many attributes & operations
 - Controller class with simple, data-object classes.
 - Lack of OO design.
 - A migrated legacy design
- **Consequences**
 - Lost OO advantage
 - Too complex to reuse or test.
 - Expensive to load



MITRE

Long Method

- Long Method is a method that is too long, so it is difficult to understand, change, or extend
- Still, no hard and fast rules about size.
- Cyclomatic Complexity metrics can be used to detect long methods from code.
- Detection rule may be:
 - ✓ more than or equals to a specified number of Lines of code (excluding method declaration, comments and parenthesis) or
 - ✓ more than or equals to a specified number of Operands and (or) Operators



Data Class

- These are Classes that have fields, getting and setting methods and nothing else
 - Is just acting as a data holder
 - Can indicate the design is really procedural
 - Detection rules will be as:
 - ✓ If the class contains public attributes and no methods the smell is detected;
 - ✓ If the class contains public attributes, setters and/or getters the smell is detected;
 - ✓ If the class contains only get and set methods the smell is detected;
-

Duplicate Code

- Percentage of duplicate code lines in the systems
- Duplicate methods in subclasses
- Duplicate expressions in same class
- Duplicate expressions in different classes

Example:

```
sqrt(pow(loc1.getX() - loc2.getX(), 2)+  
      pow(loc1.getY() - loc2.getY(), 2))
```

```
sqrt(square(loc1.getX() - loc2.getX())+  
      square(loc1.getY() - loc2.getY()))  
double square(double d) {  
    return pow(d, 2);  
}
```

Duplicate Code

[432] NS_IMETHODIMP	[467] NS_IMETHODIMP	[497] NS_IMETHODIMP
[433] LocationImpl::GetPathname(nsString	[468] LocationImpl::SetPathname(const nsString	[498] LocationImpl::GetPort(nsString& aPort)
[434] {	[469] {	[499] {
[435] nsAutoString href;	[470] nsAutoString href;	[500] nsAutoString href;
[436] nsIURI *url;	[471] nsIURI *url;	[501] nsIURI *url;
[437] nsresult result = NS_OK;	[472] nsresult result = NS_OK;	[502] nsresult result = NS_OK;
[438]	[473]	[503]
[439] result = GetHref(href);	[474] result = GetHref(href);	[504] result = GetHref(href);
[440] if (NS_OK == result) {	[475] if (NS_OK == result) {	[505] if (NS_OK == result) {
[441] #ifndef NECKO	[476] #ifndef NECKO	[506] #ifndef NECKO
[442] result = NS_NewURL(&url, href);	[477] result = NS_NewURL(&url, href);	[507] result = NS_NewURL(&url, href);
[443] #else	[478] #else	[508] #else
[444] result = NS_NewURI(&url, href);	[479] result = NS_NewURI(&url, href);	[509] result = NS_NewURI(&url, href);
[445] #endif // NECKO	[480] #endif // NECKO	[510] #endif // NECKO
[446] if (NS_OK == result) {	[481] if (NS_OK == result) {	[511] if (NS_OK == result) {
[447] #ifndef NECKO	[482] char *buf = aPathname.ToNewCString();	[512] aPort.SetLength(0);
[448] char* file;	[483] #ifndef NECKO	[513] #ifndef NECKO
[449] result = url->GetPath(&file);	[484] url->SetPath(buf);	[514] PRInt32 port;
[450] #else	[485] #else	[515] (void)url->GetPort(&port);
[451] const char* file;	[486] url->SetFile(buf);	[516] #else
[452] result = url->GetFile(&file);	[487] #endif	[517] PRUint32 port;
[453] #endif	[488] SetURL(url);	[518] (void)url->GetHostPort(&port);
[454] if (result == NS_OK) {	[489] delete[] buf;	[519] #endif
[455] aPathname.SetString(file);	[490] NS_RELEASE(url);	[520] if (-1 != port) {
[456] #ifdef NECKO	[491] }	[521] aPort.Append(port, 10);
[457] nsCRT::free(file);	[492] }	[522] }
[458] #endif	[493]	[523] NS_RELEASE(url);
[459] }	[494] return result;	[524] }
[460] NS_IF_RELEASE(url);	[495] }	[525] }
[461] }	[496]	[526]
[462] }		[527] return result;
[463]		[528] }
[464] return result;		[529]
[465] }		
[466]		

Code Clones

Type-1 Code Clone: Identical code fragments except for differences in whitespace, layout, and comments.

Type-2 Code Clone: Code fragments that are identical except for variations in identifier names and literal values, in addition to differences in whitespace, layout, and comments

Type-3 Code Clone: Code fragments with further modifications such as changes in statements, which may involve adding or removing statements, but retain similar functionality.

Type-4 Code Clone: Code fragments that are semantically similar but syntactically different. This means they perform the same functionality but are written in different ways.

Types of Clones

```
private long factorial(long n){  
    long sum = 1;  
    for(int i=1;i<=n;i++){  
        sum *= i;  
    }  
    return sum; //return  
} // Original
```

```
private long factorial(long  
n){  
    long sum = 1;  
    for(int i=1;i<=n;i++){  
        sum *= i;  
    }  
    return sum;  
} // Type-1
```

```
private long factorial(long  
num){  
    long sum = 1;  
    for(int i=1;i<=num;i++){  
        sum *= i;  
    }  
    return sum;  
} // Type-2
```

```
public static int get_factorial(int  
num){  
    int sum = 1;  
    for(int i=num;i>=1;i--){  
        sum = sum *i;  
    }  
    return sum;  
} // Type-3
```

```
public static int get_factorial(int  
n){  
    if(n == 1){  
        return 1;  
    }else {  
        return n*get_factorial(n-1);  
    }  
} // Type-4
```



Message Chain

- Long list of method calls:
 - Can be analyzed by Measuring the couplings of a method
 - `customer.getAddress().getState()`
 - `window.getBoundingBox().getOrigin().getX()`

 - Replace with shorter calls:
 - `customer.getState()`
 - `window.leftBoundary()`
-

Long Parameters List

- A long parameter list exists if
- a method contains more than or equals to a specified number of parameters. The specific (threshold) value for the number of parameters is given by the user or ; (PMD uses 10 as specified threshold while for Check Style it is 7)
- if it contains two or more (specified numbers of) parameter of same primitive data type.

Research on code smells



Very active research area



There are many studies published on detection and refactoring of code smells



Due to large body of knowledge, there are many published surveys, mapping studies and systematic review studies on the topic



There is a recent secondary study

Research in Code Smells

Table 8. Source of Data (Benchmark Projects) Used to Create Code Smell Datasets

Study	Source projects	Types	Language
S1	Apache ant, apache cassandra, apache derby, apache hadoop, apache hbase, apache hive, apache incubating, apache ivy, apache karaf, apache lucene, apache nutch, apache pig, apache qpid, apache struts, apache wicket, apache xerces, argo uml, atunes, eclipse core, elasticsearch, freemind, hibernate, hsqldb, jboss, jedit, jfreechart, jhotdraw, jsl, jvlt, sax	Open source	Java
S2	Apache ant, apache cassandra, apache derby, apache hadoop, apache hbase, apache hive, apache incubating, apache ivy, apache karaf, apache lucene, apache nutch, apache pig, apache qpid, apache struts, apache wicket, apache xerces, argo uml, atunes, eclipse core, elasticsearch, freemind, hibernate, hsqldb, jboss, jedit, jfreechart, jhotdraw, jsl, jvlt, sax	Open source	Java
S3	Apache ant, apache tomcat, jedit, android api (framework-opt-telephony), android api (frameworks-base), android api (frameworks-support), android api (sdk), android api (tool based), apache commons lang, apache cassandra, apache commons codec, apache derby, eclipse core, apache james mime4j, google guava, aardvark, amd engine, apache commons io, apache commons logging, mongo db	Open source	Java
S4	74 projects from qualitas corpus	Open source	Java
S5	76 projects from qualitas corpus	Open source	Java
S6	281 github projects	Open source	Java
S7	74 projects from qualitas corpus	Open source	Java
S8	Acuumulo ambary, atlas, aurora, batik, beam, cocoon, commons bcel, commons beanutils, commons cli, commons codec, commons collections, commons configuration, commons daemon, commons dbcp, commons dbutils, commons digester, commons exec, commons file upload, commons	Industrial	Java

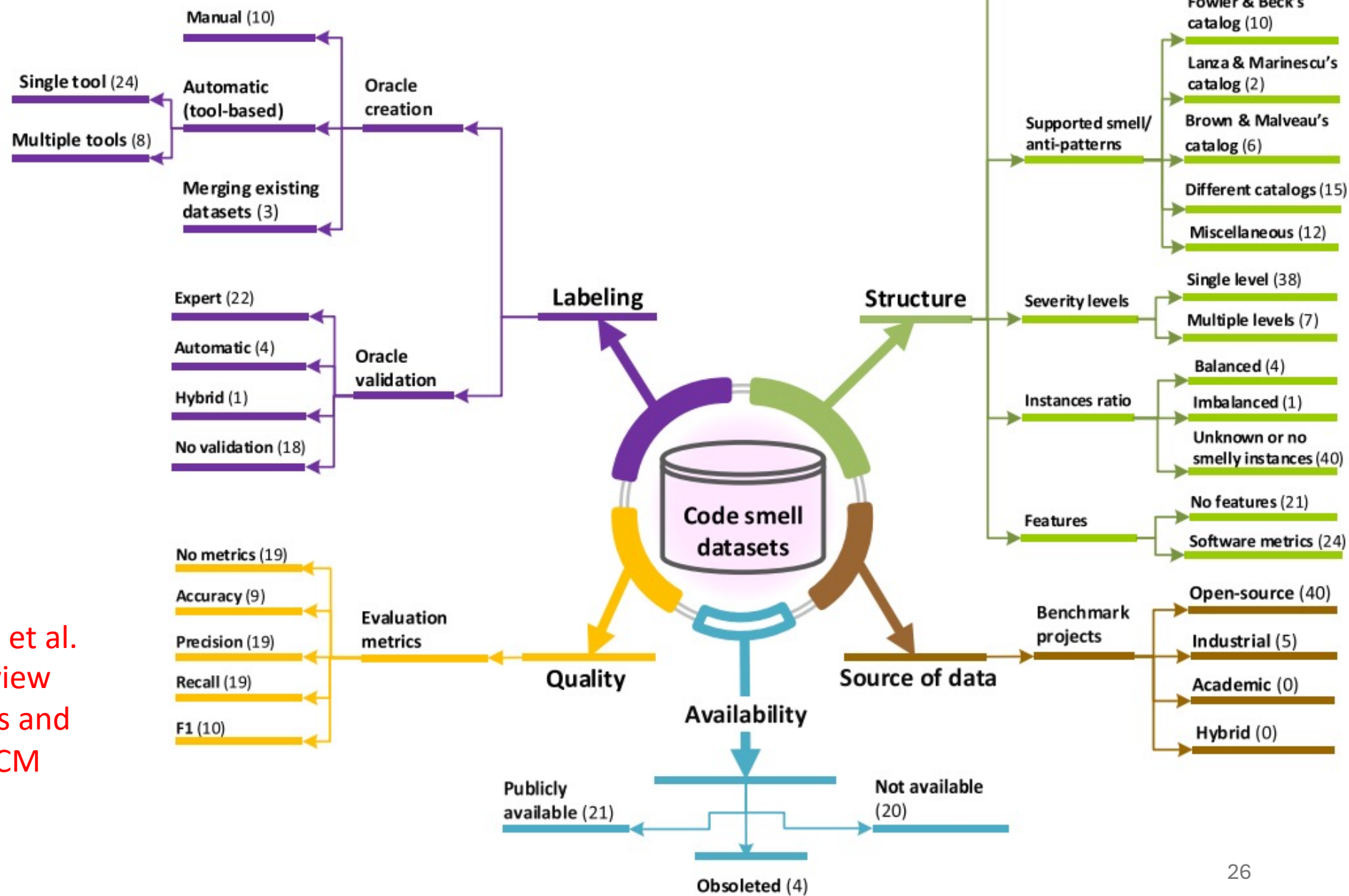
Zakeri-Nasrabadi, Morteza, et al. "A systematic literature review on the code smells datasets and validation mechanisms." ACM Computing Surveys 55.13s (2023): 1-48.

Table 9. List of Publicly Available Code Smell Datasets

Study	Status	Projects	Supported smells	Other important properties	Download link
S1	available	30	14	• 17,350 code smell instances, • .csv files	https://dibt.unimol.it/staff/fpalomba/reports/badSmell-analysis/index.html
S2	available	30	14	• 40,888 code smell instances, • .csv files	http://www.mediafire.com/file/mzyr95cgmrby19/dataset.zip/file
S3	obsolete	20	5	243 code smell instances	https://www.sesa.unisa.it/landfill/
S4	available	74	6	• 1,986 code smell and non-smell instances, • 61 metrics for class and 82 metrics for method, • .csv files	http://essere.disco.unimib.it/reverse/MLCS D.html
S5	available	76	4	• 1,986 code smell and non-smell instances • 63 metrics for class smells and eight metrics for method smells, • 4 severity levels, • .csv files	http://essere.disco.unimib.it/reverse/MLCS D.html
S6	available	281	1	• Long method instances, • PMD, Iplasma, Marinescu, Designite output for instance, • .csv files	http://madeyski.e-informatyka.pl/download/GrodzickaEtAl19DataSet.zip
S7	available	74	4	• 4 datasets each 840 instances, • 140 with specific smell and 700 without it, • 61 class metrics and 82 method metrics, • .csv files	https://figshare.com/articles/dataset/Detecting_Code_Smells_using_Machine_Learning_Techniques_Are_We_There_Yet_/5786631
S8	available	30	23	• 37,553 code smell instances, • 1,840,217 technical debts, • 5 severity levels, • 30 metrics, • .db file	https://github.com/clowee/The-Technical-Debt-Dataset
S9	available	74	2	• 445 instances of multi-label smells and non-smells, • 2 labels, • 4 sets of labels, • 46 metrics, • .csv files	https://github.com/thiru578/Multilabel-Dataset
S10	available	523	4	• 14,739 instances, • 3,291 code smell instances • 4 severity levels, • .xlsx files	https://zenodo.org/record/3666840#.YPbTXugzblU
S12	available	13	5	• 8,534 code smell instances and non-smell instances, • Naive Bayes and Décor outputs on instances, • .csv files	https://figshare.com/s/9d086c96eb1e58350f3d
S19	available	8	2	• 262 smelly samples, • 11 metrics, • .txt files	https://github.com/antoineBarbez/SMAD
S22	available	3	4	• 3,162 instances, • 50 metrics, • .csv and .arrf files	http://www.ptidej.net/download/experiments/wcre12a
S23	available	2	3	• 777 instances, • 147 smell instances, • Décor output, • Metrics, • .xls files	http://www.ptidej.net/downloads/experiments/jss10

Zakeri-Nasrabadi, Morteza, et al. "A systematic literature review on the code smells datasets and validation mechanisms." *ACM Computing Surveys* 55.13s (2023): 1-48.

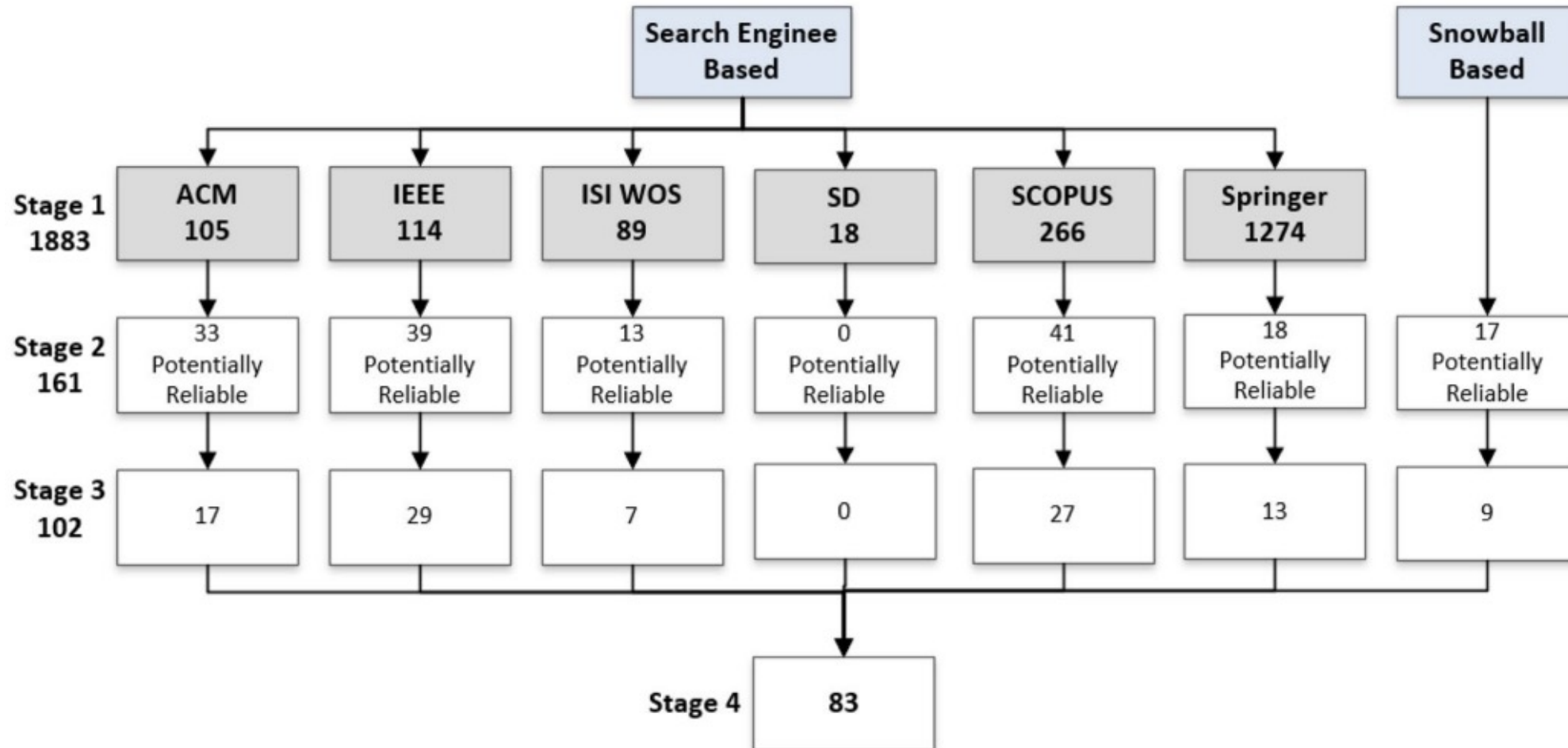
Classification of code smell datasets.



Zakeri-Nasrabadi, Morteza, et al.
 "A systematic literature review
 on the code smells datasets and
 validation mechanisms." ACM
 Computing Surveys 55.13s
 (2023): 1-48.

Research in Code Smells

Pereira dos Reis, José, et al. "Code smells detection and visualization: a systematic literature review." *Archives of Computational Methods in Engineering* 29.1 (2022): 47-94.



Research in Code Smells

Pereira dos Reis, José,
et al. "Code smells
detection and
visualization: a
systematic literature
review." *Archives of
Computational Methods
in Engineering* 29.1
(2022): 47-94.

Fig. 3 Trend of publication years

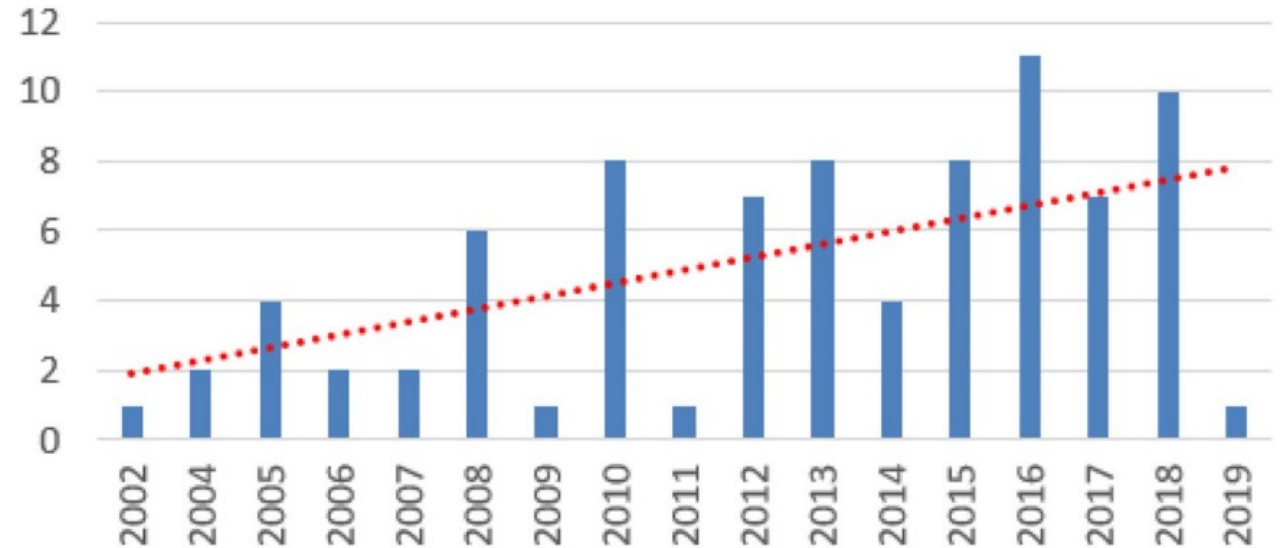


Fig. 4 Type of publication venue

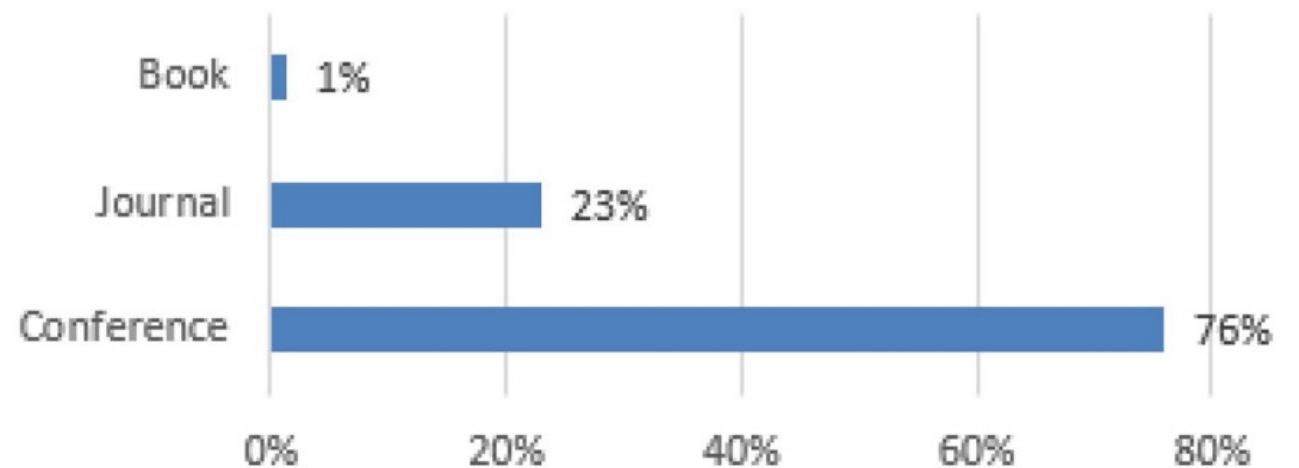
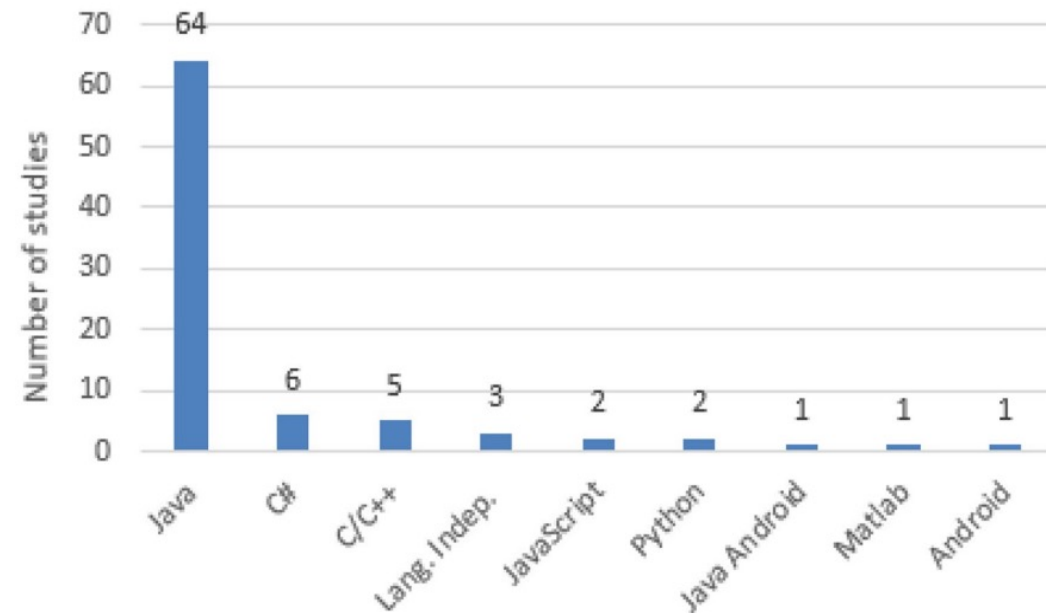


Table 7 CS detection approaches used

Approaches	No. of studies	% Studies	Studies
Search-based	25	30.1	S5, S6, S14, S22, S24, S28, S33, S34, S36, S37, S42, S43, S45, S48, S51, S53, S55, S56, S71, S74, S75, S77, S78, S79, S83
Metric-based	20	24.1	S3, S9, S29, S38, S44, S47, S49, S52, S58, S60, S63, S64, S66, S67, S68, S70, S72, S73, S81, S82
Symptom-based	16	19.3	S4, S8, S13, S15, S20, S23, S26, S30, S31, S32, S52, S57, S59, S61, S62, S69
Visualization-based	12	14.5	S1, S2, S7, S11, S12, S16, S17, S19, S21, S39, S46, S76
Probabilistic	10	12.0	S10, S18, S25, S27, S35, S40, S50, S54, S65, S80
Cooperative-based	1	1.2	S41
Manual	1	1.2	S52

Programming languages and number of studies that use them



Conclusion



Software Refactoring and Maintenance is heavily based on the detection of Code Smells.



Detection tools for the detection of different Code Smells from multiple languages is missing.



There is disparity in results of different tools



Standard benchmarks are highly desired



The impacts of Code Smells is heavily based on accurate detection of all different code smells.



More focus is required on refactoring techniques and tools

Thanks for your attention!

Any Question?



Email me on : junaidakram@cuilahore.edu.pk